

Zalord — Hệ thống Chat Microservices

Kiến trúc & Triển khai

Nhóm 17

Trường Đại học Công nghệ Thông tin — ĐHQG TP.HCM
GVHD: TS. Nguyễn Duy Khánh

Tháng 6, 2026

Nội dung trình bày

- 1 Tổng quan hệ thống
- 2 Code Organization
- 3 Domain-Driven Design
- 4 Nguyên lý thiết kế Microservices
- 5 Giao tiếp giữa các dịch vụ
- 6 Thiết kế & Quản lý cơ sở dữ liệu
- 7 Triển khai Docker & Kubernetes
- 8 API Gateway
- 9 Authentication & Authorization
- 10 Logging, Monitoring, Tracing
- 11 CI/CD & Deployment
- 12 Kết luận

1. Tổng quan hệ thống — Giới thiệu Zalord

Zalord là gì?

- Ứng dụng chat real-time kiến trúc **Microservices**
- **7 services** độc lập, polyglot Java / Go
- Hỗ trợ: nhắn tin 1-1, nhóm, file đính kèm, real-time presence,...
- Frontend: React 19 + TypeScript + Vite + WebSocket

Tech stack tổng quan

- **Gateway:** Kong 3.7
- **Messaging:** RabbitMQ / Kafka (switchable)
- **Storage:** PostgreSQL ×6, Redis, MinIO (S3)
- **Observability:** Prometheus + Grafana
- **Deploy:** Docker Compose (local) / k3s VPS (stag) / GKE (prod)
- **CI/CD:** GitHub Actions → Docker Hub → GKE / k3s

1. Tổng quan hệ thống — Danh sách Microservices

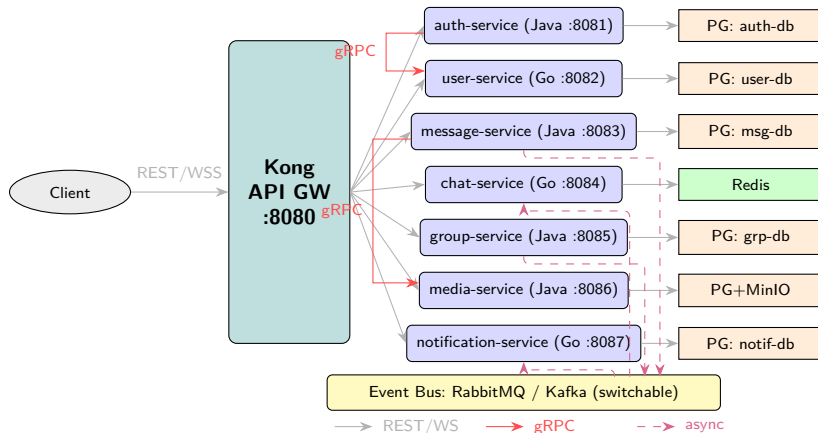
Service	Lang	Port
auth-service	Java/Spring	8081
user-service	Go	8082
message-service	Java/Spring	8083
chat-service	Go	8084
group-service	Java/Spring	8085
media-service	Java/Spring	8086
notification-service	Go	8087

gRPC Internal (không qua Kong)

auth → user (CreateProfile :9082)

message → media (ValidateAttachments :9086)

1. Tổng quan hệ thống — Kiến trúc tổng quan



2. Code Organization — Why Layered Architecture?

Tại sao chọn Layered Architecture thay vì Clean Architecture hay Vertical Slice Architecture?

Tiêu chí	Layered (Đang dùng)	Clean / Hexagonal	Vertical Slice (VSA)
Số file / tính năng	3 file (Controller, Service, Repo)	5-7 file (Port, Adapter, Use-case, Entity, DTO)	3-4 file (Request, Response, Handler inline)
Share Dependencies (VD: Config, Repo)	Dễ dàng tiêm 1 lần vào chung class Service	Dễ dàng tiêm vào chung Use case class	Khó — mỗi slice bị cô lập, phải tự tiêm lại từ đầu
Độ tương thích với Framework	Tuyệt đối (Hưởng lợi từ Spring annotations)	Xung đột (Domain không được chứa thư viện Spring/JPA)	Kém (Spring không hỗ trợ MediatR built-in như .NET)
Độ phức tạp & Duplication	Thấp, code base đơn giản, tập trung	Boilerplate quá cao so với quy mô dự án 3 tháng	Trùng lặp code nhiều (VD: Outbox pattern phải copy)

Kết luận cho bài toán thực tế của Zalord

Team 3 người, thời hạn 3 tháng, service nhỏ (~8-10 endpoints).

→ **Layered Architecture** mang lại tốc độ code nhanh nhất, tận dụng tối đa tiện ích của Spring Boot, không bị "over-engineering".

3. Domain-Driven Design — Tại sao không dùng Modulithic?

Lý do chọn Microservices

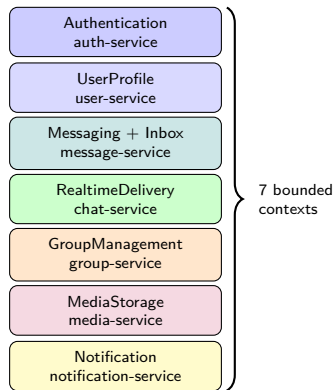
- **Polyglot language:** chat-service dùng Go (goroutines) cho WebSocket fan-out hiệu năng cao; business logic dùng Java Spring Boot
- **Independent scaling:** chat-service cần scale riêng khi traffic tăng, không kéo theo auth-service
- **Independent deploy:** cập nhật media-service không ảnh hưởng chat real-time
- **Fault isolation:** media-service lỗi không kéo sập toàn bộ hệ thống

Nếu là Modulithic...

- Không thể mix Go + Java trong 1 process
- Scale toàn bộ khi chỉ cần scale 1 module
- Deploy toàn bộ khi chỉ thay đổi 1 module
- 1 memory leak / goroutine leak làm sập tất cả

3. Domain-Driven Design — Phân chia theo DDD - Bounded Contexts

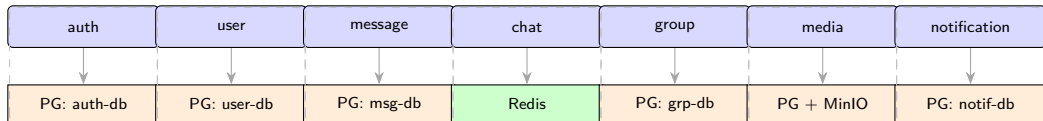
Service	Bounded Context	Aggregate Root
auth-service	Authentication	Account, Session
user-service	UserProfile	UserProfile
message-service	Messaging + Inbox	Message, ConversationView
chat-service	RealtimeDelivery	Connection
group-service	GroupManagement	Group
media-service	MediaStorage	MediaItem
notification-service	Notification	Notification



3. Domain-Driven Design — DB per Bounded Context

Nguyên tắc phân chia

- Mỗi service có **1 bounded context** độc lập
- Không share entity/table giữa các service
- Giao tiếp qua **API / Event** (không gọi thẳng DB)
- conversation.id được tái sử dụng làm group.id (shared key, không shared schema)



Mỗi service chỉ truy cập database của chính mình — không share schema

Redis (shared): auth sessions, chat presence/pub-sub, message membership cache

MinIO (shared): avatars, attachments

4. Nguyên lý thiết kế Microservices — Single Responsibility Principle

SRP — Mỗi service 1 trách nhiệm rõ ràng

- **chat-service**: CHỈ quản lý WebSocket connections + fan-out messages → không write business DB, không có business logic
- **media-service**: CHỈ quản lý upload lifecycle (generate presigned URL → MinIO → finalize metadata)
- **notification-service**: CHỈ consume events + persist + serve notification list
- **auth-service**: CHỈ authentication (issue/validate token, session management)

4. Nguyên lý thiết kế Microservices — High Cohesion & Low Coupling

High Cohesion — Low Coupling

- **High Cohesion:** dữ liệu và logic cùng domain nằm trong 1 service, không scatter
- **Low Coupling:** chỉ **2 synchronous dependencies** (gRPC):
 - auth → user (registration)
 - message → media (validate attachments)
- Mọi giao tiếp còn lại: **async event bus**
- Downstream services không parse JWT — trust header từ Kong

Execution Pipeline

Request → Kong (validate) → Service → DB / Event Bus

Không có cross-service synchronous chain dài hơn 2 hops

5. Giao tiếp giữa các dịch vụ — Sync vs Async - Khi nào dùng gì?

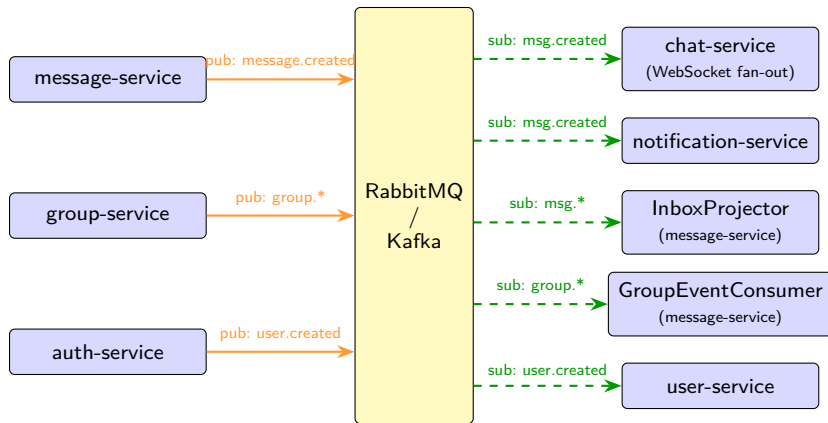
Synchronous — gRPC (2 trường hợp)

- **auth** → **user-service** (CreateProfile):
 - Cần response ngay để rollback nếu profile creation fail
 - Deadline 5 giây, circuit breaker Resilience4j
- **message** → **media-service** (ValidateAttachments):
 - Validate media IDs trước khi persist message
 - Deadline 3 giây (hot path user-facing)
- Dùng gRPC vì: type-safe contract (Protobuf), binary encoding nhanh hơn REST, không qua Kong (internal network)

Asynchronous — Event Bus (Kafka / RabbitMQ)

- **message.created** → chat-service (WebSocket fan-out), notification-service, InboxProjector
- **group.*** (created/member-added/removed/updated) → message-service (project Conversation)
- **user.created** → user-service
- Dùng async vì: fan-out 1-to-many, producers không cần chờ consumers, failure isolation
- **Switchable**: EVENT_BUS=rabbitmq hoặc kafka — cùng interface EventBus (Factory pattern)

5. Giao tiếp giữa các dịch vụ — Event-Driven Architecture - Message Flow



5. Giao tiếp giữa các dịch vụ — Benchmark: Kafka vs RabbitMQ

Kết quả Benchmark

- **Kafka:** Throughput cao hơn (hàng trăm ngàn msg/s), phù hợp cho log aggregation và event sourcing.
- **RabbitMQ:** Độ trễ (latency) thấp hơn ở mức tải trung bình, hỗ trợ routing linh hoạt (topic exchange, dead-letter queue).

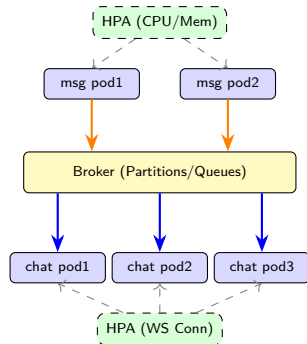
[PLACEHOLDER — biểu đồ benchmark]

Biểu đồ so sánh Throughput & Latency giữa Kafka và RabbitMQ

5. Giao tiếp giữa các dịch vụ — Scaling cho luồng gửi Message

Chiến lược Scaling

- **Producer (message-service):** Scale theo CPU/Memory (HPA) vì việc lưu DB và publish event đòi hỏi xử lý nhanh.
- **Consumer (chat-service):** Scale theo số lượng kết nối WebSocket đang mở, giới hạn số lượng connection per pod.
- **Broker:** Tăng số lượng Partition (Kafka) hoặc chia nhỏ Queue (RabbitMQ) để scale việc tiêu thụ event.



5. Giao tiếp giữa các dịch vụ — Thiết kế Endpoint

REST API — Naming Convention

- Base path: `/api/v1/{resource}` (plural noun)
- POST `/api/v1/auth/register`
- GET `/api/v1/users`
- GET `/api/v1/conversations`
- POST `/api/v1/messages`
- POST `/api/v1/groups/{id}/members`
- DELETE `/api/v1/groups/{id}/members/{uid}`
- POST `/api/v1/media/upload-url`
- WebSocket: `/ws/chat?token={jwt}`

Error Response

Chuẩn **RFC 7807**
(Platform-agnostic):

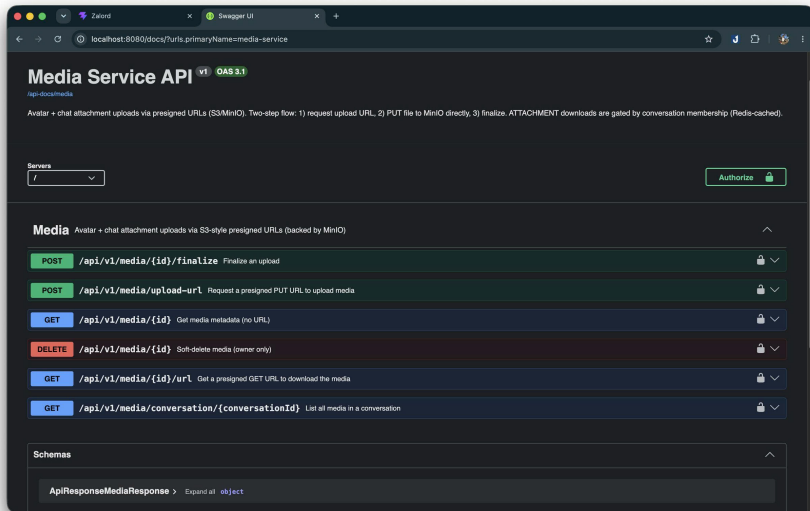
```
{"type":...,  
  "status":400,  
  "detail":"...",  
  "title":"..."}
```


5. Giao tiếp giữa các dịch vụ — Swagger / OpenAPI Aggregation

Swagger / OpenAPI Aggregation

- Java services: **springdoc-openapi** → /v3/api-docs
- Go services: **swaggo** → /v3/api-docs
- Kong route /api-docs/{svc} → strip_path: true → forward đến service /v3/api-docs
- **Swagger UI** container tại /docs aggregate tất cả

5. Giao tiếp giữa các dịch vụ — Swagger / OpenAPI Aggregation



6. Thiết kế & Quản lý cơ sở dữ liệu — DB per Service & Consistency

DB per Service

- 6 PostgreSQL instances độc lập + Redis + MinIO
- Không service nào truy cập trực tiếp DB của service khác
- **Strong consistency**: trong 1 service (ACID transactions)
- **Eventual consistency**: cross-service qua event bus
 - group member change → message-service projection có độ trễ nhỏ
 - message.created → inbox view update không đồng thời

Consistency cần thiết ở đâu?

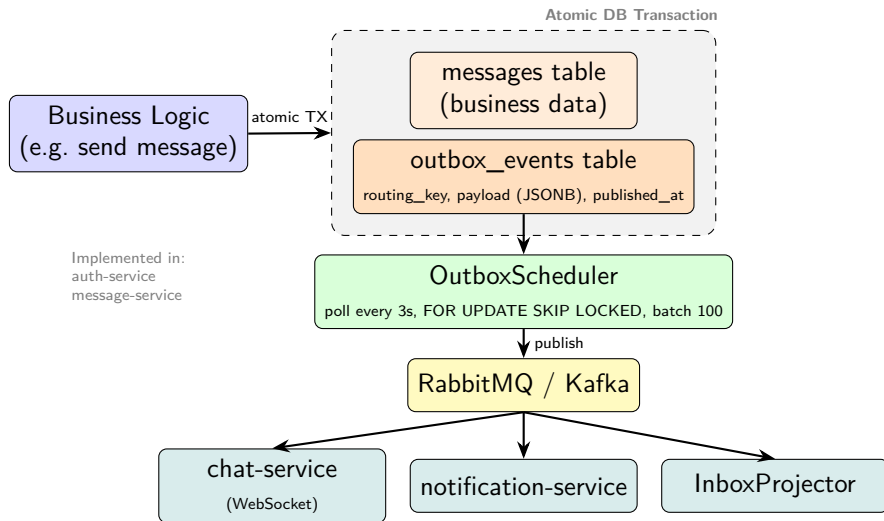
- **Strong**: auth (login/register), message persist (không mất tin nhắn)
- **Eventual**: inbox unread count, notification, presence status

6. Thiết kế & Quản lý cơ sở dữ liệu — CAP Theorem

CAP Theorem

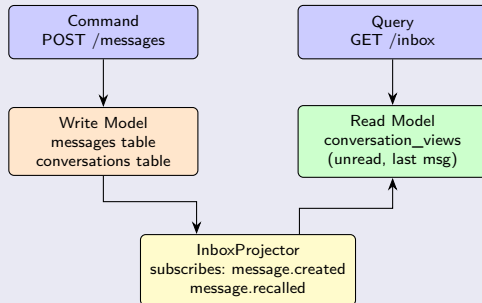
- PostgreSQL: **CP** (Consistency + Partition Tolerance)
- Chấp nhận availability giảm khi partition
- Redis: **AP** — eventual consistency cho presence data

6. Thiết kế & Quản lý cơ sở dữ liệu — Transactional Outbox Pattern



6. Thiết kế & Quản lý cơ sở dữ liệu — CQRS trong message-service

CQRS trong message-service

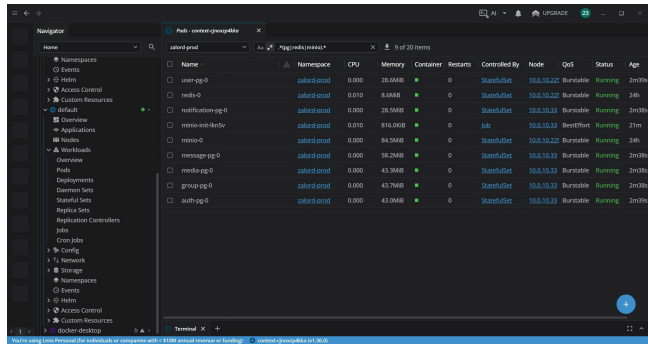


projection có độ trễ nhỏ (eventual consistency)

6. Thiết kế & Quản lý cơ sở dữ liệu — Polyglot Persistence

Polyglot Persistence

Store	Dùng cho
PostgreSQL ×6	Business data (ACID)
Redis	Sessions, presence, pub/sub
MinIO (S3)	Avatars, attachments



7. Triển khai Docker & Kubernetes — Dockerfile

Multi-Stage Dockerfile — Java (auth-service)

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-jammy AS build
WORKDIR /workspace
COPY mvnw . && COPY .mvn .mvn
COPY pom.xml .
RUN chmod +x mvnw && ./mvnw -B dependency:go-offline
# Layer cache: deps downloaded before source copy
COPY src src
RUN ./mvnw -B -DskipTests clean package \
    && java -Djarmode=layertools -jar target/*.jar \
        extract --destination target/extracted

# Stage 2: Runtime (JRE only, alpine = small)
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring
COPY --from=build /workspace/target/extracted/ ./
EXPOSE 8081
```

Multi-Stage Dockerfile — Go (chat-service)

```
# Stage 1: Build
FROM golang:1.26.1-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download          # deps cached separately
COPY . .
RUN CGO_ENABLED=0 go build -o server ./cmd/server

# Stage 2: Runtime (scratch-like, ~8MB)
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/server .
EXPOSE 8084
CMD ["/server"]
```


7. Triển khai Docker & Kubernetes — Docker Compose

Docker Compose

- 20+ containers, 1 network zalord-net
- Named volumes per DB (pgdata_auth, ..., miniodata, redisdata)
- Env vars từ .env file, không hardcode

7. Triển khai Docker & Kubernetes — Kubernetes / k3s

Kustomize Overlays

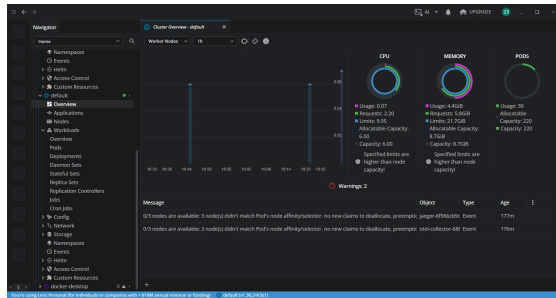
- Path: `deploy/k8s/overlays/dev/`
- Namespace: `zalord-dev`
- **Workloads:** 7 Deployments (services)
- **StatefulSets:** Kafka, RabbitMQ, Redis, PostgreSQL×6, MinIO (kèm Persistent Volumes 1–5Gi)
- **Resources:** req/limit (e.g. `auth: 100m/384Mi req`)
- **Probes:** TCP liveness, HTTP `/health`
- **Secrets:** SOPS + Age encryption

```
zalord@zalord:~$ kubectl get pods -n zalord-dev
No resources found in zalord-dev namespace.
zalord@zalord:~$ kubectl get pods -n zalord-staging
NAME                                READY   STATUS    RESTARTS   AGE
auth-pg-0                           1/1     Running   0           18h
auth-service-5d4b4d4d-cv7n4         1/1     Running   0           18h
chat-service-577d4d4d95-r46zh       1/1     Running   4 (3h14m ago)  3h14m
chat-service-577d4d4d95-tzmxp       1/1     Running   6 (5h0m ago)   5h12m
frontend-0d0b4d4d-cd5u8             1/1     Running   0           18h
grafana-57b4d4d4-7y9rb              1/1     Running   0           18h
group-pg-0                           1/1     Running   0           18h
group-service-9d57d4d48-9w5g        1/1     Running   0           18h
jaeger-4f96d4d48-cv5fn              0/1     Pending   0           3h2m
jaeger-7fcd4d4d78-3qg7f             1/1     Running   40 (169m ago)  18h
kafka-0                              1/1     Running   1 (13h ago)    3h1m
media-pg-0                           1/1     Running   0           18h
media-service-5d4d9d9d4-wfcb        1/1     Running   0           18h
message-pg-0                         1/1     Running   0           3h1m
message-service-3b43d4d48-2b348     1/1     Running   0           3h14m
message-service-3b43d4d48-4b4c6     1/1     Running   2 (3h13m ago)  3h14m
notification-pg-0                   1/1     Running   0           18h
notification-service-6d43d4d48-822e 1/1     Running   6 (5h0m ago)   5h12m
notification-service-6d43d4d48-j4g5 1/1     Running   4 (3h14m ago)  3h14m
otel-collector-5d7d9d9d4-cv48c      1/1     Running   25 (17h ago)   18h
otel-collector-6b4d4d48-b4dcb       0/1     Pending   0           3h1m
prometheus-6d5d7c8d79-up4mf         1/1     Running   0           18h
rabbitmq-0                          1/1     Running   0           8h2m
redis-0                             1/1     Running   0           18h
user-pg-0                           1/1     Running   0           18h
user-service-df4b4d48-4b4hr         1/1     Running   1 (18h ago)    18h
zalord@zalord:~$
```

7. Triển khai Docker & Kubernetes — Kubernetes Lens

K8s Lens / OpenLens

- Sử dụng **K8s Lens** làm công cụ quản lý cluster trực quan (GUI).
- Dễ dàng theo dõi trạng thái Pods, Nodes, Deployments, và StatefulSets.
- Hỗ trợ xem logs, exec vào container, và chỉnh sửa YAML trực tiếp không cần thao tác dòng lệnh.



7. Triển khai Docker & Kubernetes — Service Discovery

Service Discovery — K8s DNS

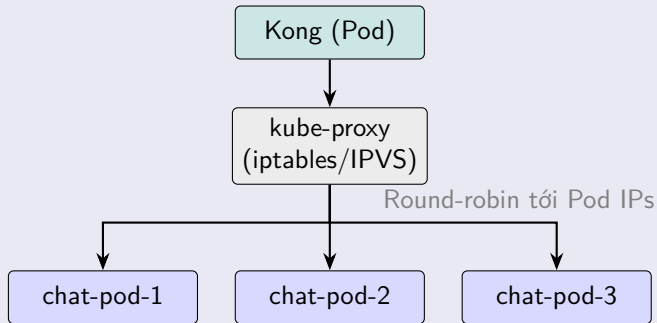
- K8s tự tạo DNS entry cho mỗi ClusterIP Service
- `auth-service.zalord-dev.svc.cluster.local`
- Các service gọi nhau qua tên service nội bộ (e.g. `http://user-service:8082`)
- **KHÔNG cần Eureka hay Consul** — K8s DNS thay thế hoàn toàn

Cấu hình gRPC target

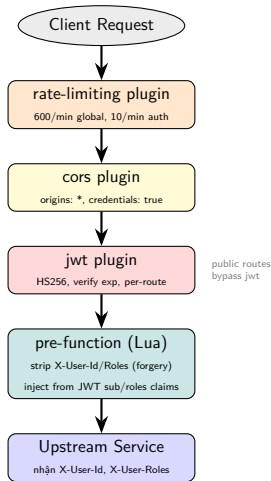
- `ZALORD_USER_SERVICE_GRPC_TARGET=user-service:9082`
- `ZALORD_MEDIA_SERVICE_GRPC_TARGET=media-service:9086`
- Quá trình: DNS resolve → ClusterIP → Pod IP

7. Triển khai Docker & Kubernetes — Load Balancing

Load Balancing — Server-side



8. API Gateway — Kong - Filter Chain



Các tính năng Kong đã cấu hình

- **Rate Limiting:** 600 req/min per IP (global), 10 req/min (auth routes /login, /register)
- **CORS:** global, tất cả origins (dev), credentials enabled
- **JWT Filter:** validate HS256, kiểm tra exp claim
- **Identity Injection:** Lua pre-function inject X-User-Id (từ sub) và X-User-Roles (từ roles array)
- **Prometheus:** metrics tại admin :8001/metrics

8. API Gateway — Kong - Route Table

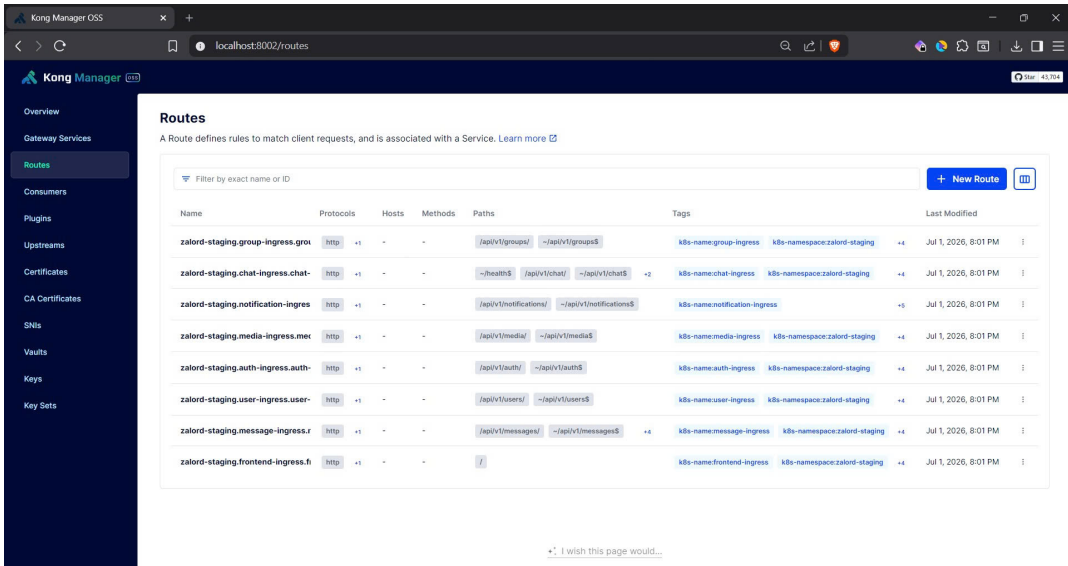
Declarative Config (DB-less)

- Config tại `infra/kong/kong.yml` (version control). **Cùng 1 file** cho Dev & Prod (tránh config drift).
- `__JWT_SECRET__` được thay bằng env var khi start — bảo mật secret.
- **KHÔNG** dùng **K8s Ingress Controller** (`networking.k8s.io/v1`). Đây là **lựa chọn thiết kế** standalone (xem `EXPLANATION.md`).

Route Table

Route	Upstream	JWT
<code>/api/v1/auth/*</code>	<code>auth:8081</code>	×
<code>/api/v1/users</code>	<code>user:8082</code>	✓
<code>/api/v1/messages</code>	<code>message:8083</code>	✓
<code>/ws/chat</code>	<code>chat:8084</code>	✓
<code>/api/v1/groups</code>	<code>group-service:8085</code>	✓
<code>/api/v1/media</code>	<code>media-service:8086</code>	✓
<code>/api/v1/notifications</code>	<code>notification-svc:8087</code>	✓
<code>/docs</code>	<code>swagger-ui:8080</code>	×

8. API Gateway — Kong - Manager UI



Kong Manager OSS

localhost:8002/routes

Kong Manager OSS

Overview

Gateway Services

Routes

Consumers

Plugins

Upstreams

Certificates

CA Certificates

SNIs

Vaults

Keys

Key Sets

Routes

A Route defines rules to match client requests, and is associated with a Service. [Learn more](#)

Filter by exact name or ID

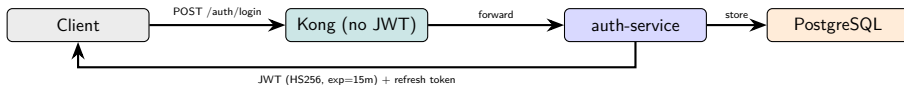
+ New Route

Name	Protocols	Hosts	Methods	Paths	Tags	Last Modified
zalord-staging.group-ingress.gro	http	+1	-	-	/api/v1/groups/ ~/api/v1/groups\$	k8s-name:group-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.chat-ingress.chat-	http	+1	-	-	~/health\$ /api/v1/chat/ ~/api/v1/chat\$ +2	k8s-name:chat-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.notification-ingres	http	+1	-	-	/api/v1/notifications/ ~/api/v1/notifications\$	k8s-name:notification-ingress +5 Jul 1, 2026, 8:01 PM
zalord-staging.media-ingress.mer	http	+1	-	-	/api/v1/media/ ~/api/v1/media\$	k8s-name:media-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.auth-ingress.auth-	http	+1	-	-	/api/v1/auth/ ~/api/v1/auth\$	k8s-name:auth-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.user-ingress.user-	http	+1	-	-	/api/v1/users/ ~/api/v1/users\$	k8s-name:user-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.message-ingress.r	http	+1	-	-	/api/v1/messages/ ~/api/v1/messages\$ +4	k8s-name:message-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM
zalord-staging.frontend-ingress.fi	http	+1	-	-	/	k8s-name:frontend-ingress k8s-namespace:zalord-staging +4 Jul 1, 2026, 8:01 PM

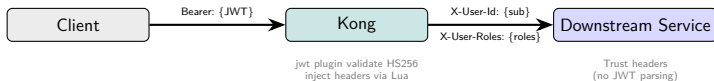
+ I wish this page would...

9. Authentication & Authorization — JWT Authentication Flow

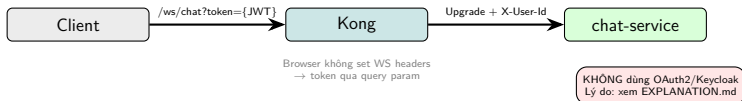
(1) Login



(2) API Call



(3) WebSocket



9. Authentication & Authorization — ConfigMap - Non-sensitive config

ConfigMap — Non-sensitive config

- **Shared:** shared-config.yaml
 - EVENT_BUS=rabbitmq
 - RABBITMQ_HOST=rabbitmq
 - KAFKA_BOOTSTRAP=kafka:9092
 - REDIS_HOST=redis
 - JWT_EXPIRY_MINUTES=15
- **Per-service:** {svc}-config.yaml — database host/port/name, gRPC targets, service-specific config
- Mount as env vars vào Deployment

Tại sao cần ConfigMap?

- Thay đổi config không cần build lại Docker image
- Khác nhau giữa environments (dev/staging/prod) qua Kustomize overlays

9. Authentication & Authorization — Secret - Sensitive data

Secret — Sensitive data (SOPS encrypted)

- **Shared:** DB passwords, JWT_SECRET, MQ credentials
- **Per-service:** database URLs với credentials
- Files: *.secret.enc.yaml — encrypted với **SOPS + Age key**
- Age key lưu trong **GitHub Actions Secret**
- CI/CD decrypt tại deploy time: `sops -d file.enc.yaml`
- Secret encrypted at rest trong Git — an toàn commit public repo

Kong JWT Secret

kong.yml chứa `__JWT_SECRET__` placeholder. Container entrypoint dùng `sed` substitute từ K8s Secret env var. Secret không bao giờ nằm trong file config.

10. Logging, Monitoring, Tracing — Prometheus & Grafana - Observability

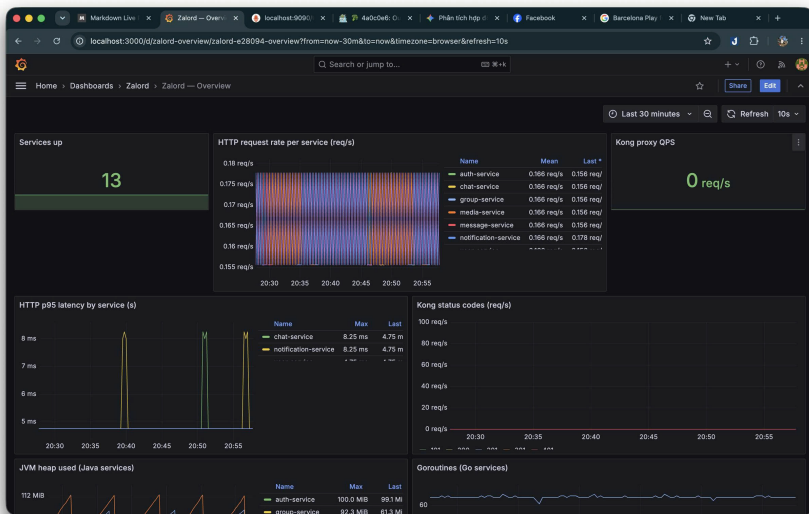
Prometheus Scrape Config

Target	Endpoint
Java Services	/actuator/prometheus
Go Services	/metrics
Kong	kong:8001/metrics
RabbitMQ	rabbitmq:15692/metrics
Kafka	kafka-exporter:9308

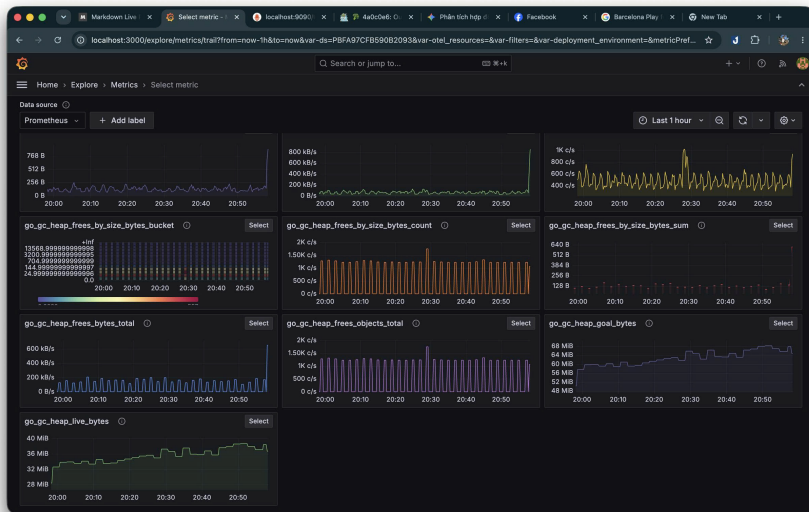
Metrics được track

- HTTP latency, request count, status codes
- Active WebSocket connections (chat)
- Message throughput, queue lag
- JVM heap, GC, Pod resource

10. Logging, Monitoring, Tracing — Prometheus & Grafana - Dashboard



10. Logging, Monitoring, Tracing — Prometheus & Grafana - Graph

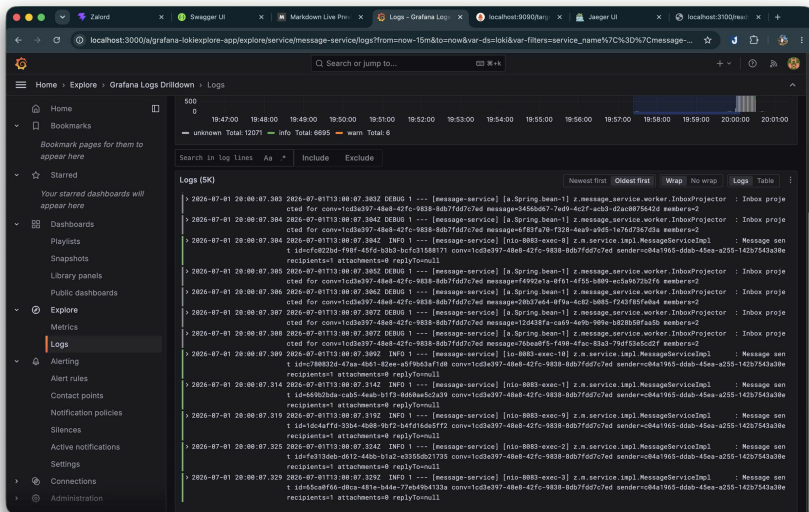


10. Logging, Monitoring, Tracing — Centralized Logging

Grafana Loki

- Thay vì dùng stack ELK nặng nề, hệ thống dùng **Loki** + **Promtail** nhẹ hơn nhiều, tối ưu cho K8s.
- Promtail tự động thu thập stdout của tất cả pods và đẩy lên Loki.
- Truy vấn log tập trung thông qua **Grafana** (LogQL). Không cần login vào từng pod bằng `kubectl logs`.

10. Logging, Monitoring, Tracing — Centralized Logging

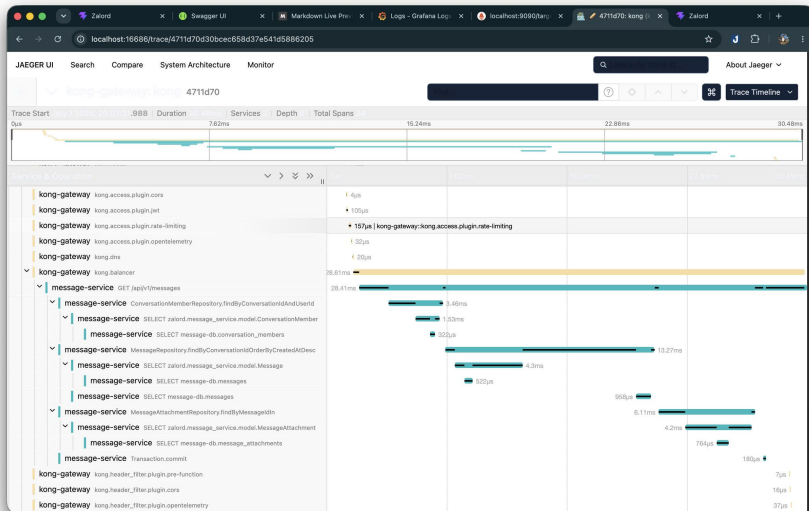


10. Logging, Monitoring, Tracing — Distributed Tracing Flow

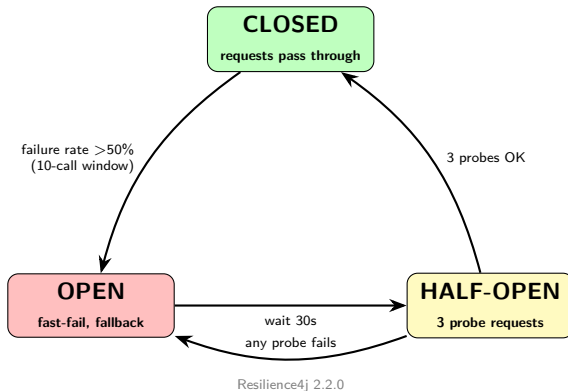
OpenTelemetry & Jaeger

- Sử dụng **OpenTelemetry (OTel)** để inject trace-id qua các HTTP header và gRPC metadata.
- **Luồng:** API Gateway → Microservice A → gRPC → Microservice B → RabbitMQ/Kafka → Consumer C.
- Dữ liệu được đẩy về **Jaeger** UI để trực quan hóa toàn bộ chuỗi request.

10. Logging, Monitoring, Tracing — Distributed Tracing Flow - Jaeger UI



10. Logging, Monitoring, Tracing — Circuit Breaker Diagram



10. Logging, Monitoring, Tracing — Timeout, Circuit Breaker & Recovery

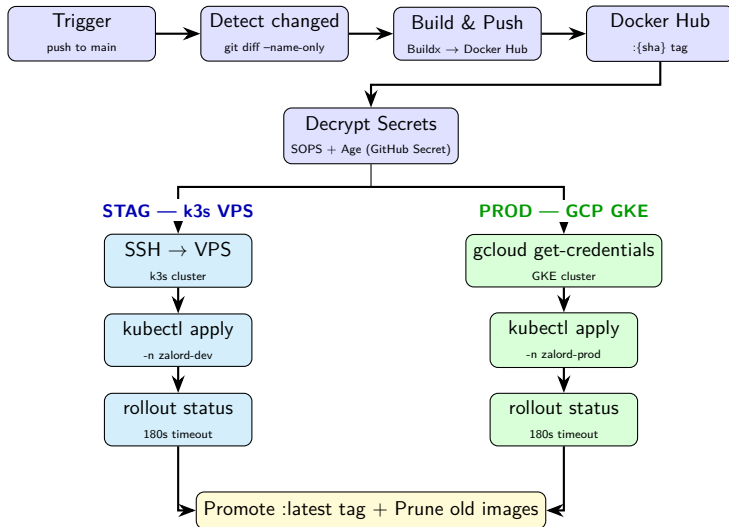
Implementation

- **auth-service** → user-service gRPC:
 - `@CircuitBreaker(name="userGrpc")`
 - gRPC deadline: **5 giây**
 - Fallback: `createProfileFallback()`
- **message-service** → media-service gRPC:
 - `@CircuitBreaker(name="mediaGrpc")`
 - gRPC deadline: **3 giây** (hot path)
 - Fallback: `validateFallback()`

Go services — Error Handling

- **PermanentError**: drop message (ack), log error
- **Transient error**: nack + requeue → RabbitMQ retry
- Frontend WebSocket: exponential backoff reconnect (cap 15s)

11. CI/CD & Deployment — GitHub Actions Pipeline



11. CI/CD & Deployment — GitHub Actions Pipeline

The screenshot shows a GitHub Actions workflow run titled "Deploy Production OKE" (run #47) for the repository "thin0704hcm/zalord". The workflow is in a "Completed" state, having succeeded 50 minutes ago in 1m 58s. The workflow file is named "deploy".

The left sidebar shows the workflow file "deploy" selected under the "Summary" tab. The main area displays the workflow steps:

- Set up job (25s)
- Checkout (1s)
- Validate required secrets (0s)
 - Download CI image metadata (0s)
- Load CI image metadata (0s)
 - Download CI image manifest (0s)
- Load CI image manifest (0s)
- Detect changed services (0s)
- Set up kubectl (1s)
- Install deploy tools (0s)
- Configure SOPS age key (0s)
- Docker Hub login (1s)
- Check image existence (2s)
- Configure OKE kubeconfig (2s)
- Build deploy bundle (0s)
- Apply manifests (24s)
- Apply canary images (5s)

The bottom of the screen shows the URL "https://github.com/pulls" and a navigation bar with icons for back, forward, and search.

11. CI/CD & Deployment — Versioning & Rollback Strategy

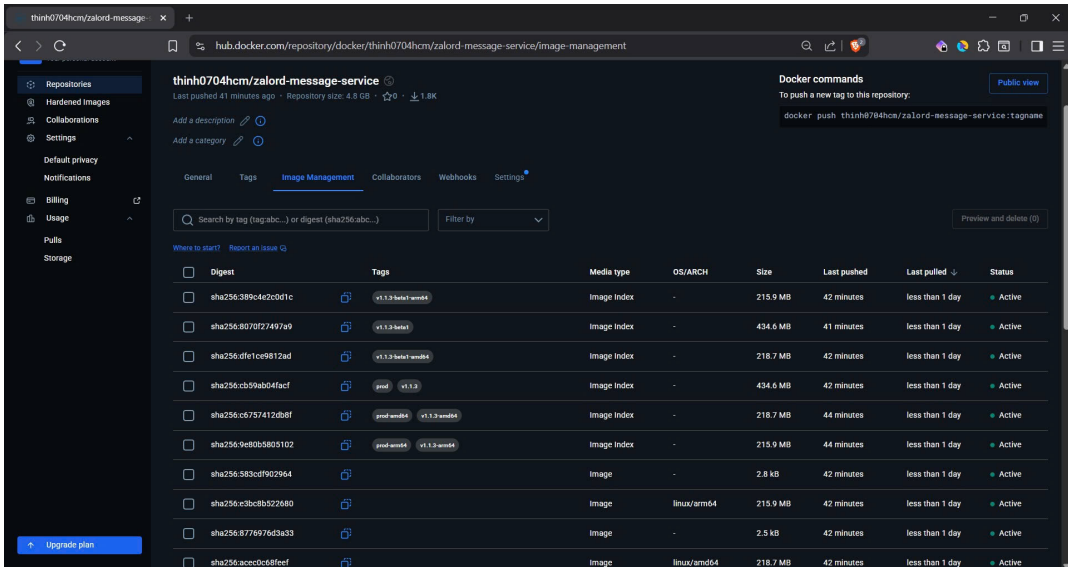
Image Versioning

- Mỗi image tagged với **Git SHA**:
 - `docker.io/{user}/zalord-auth:{sha}`
 - `docker.io/{user}/zalord-chat:{sha}`
 - ...
- Sau khi deploy thành công: promote `:latest` tag
- **Selective build**: chỉ rebuild service có file thay đổi → tiết kiệm thời gian CI
- Không có image nào bị overwrite — SHA tag immutable

Rollback

- K8s giữ revision history của Deployment
- `kubectl rollout undo ...` → về image SHA của commit trước
- `kubectl rollout status` trong pipeline → fail fast (180s) nếu pods không healthy

11. CI/CD & Deployment — Versioning & Rollback Strategy



The screenshot shows the Docker Hub interface for the repository `thinh0704hcm/zalord-message-service`. The page is in the "Image Management" tab, displaying a table of image versions. The table includes columns for Digest, Tags, Media type, OS/ARCH, Size, Last pushed, Last pulled, and Status. The repository has a size of 4.8 GB and a last push time of 41 minutes ago. The Docker commands section shows the command `docker push thinh0704hcm/zalord-message-service:tagname`.

thinh0704hcm/zalord-message-service
Last pushed 41 minutes ago · Repository size: 4.8 GB · ☆0 · 1.8K

[Add a description](#) [Add a category](#)

[General](#) [Tags](#) [Image Management](#) [Collaborators](#) [Webhooks](#) [Settings](#)

Search by tag (tag:abc...) or digest (sha256:abc...) [Filter by](#) [Preview and delete \(0\)](#)

[Where to start?](#) [Report an issue](#)

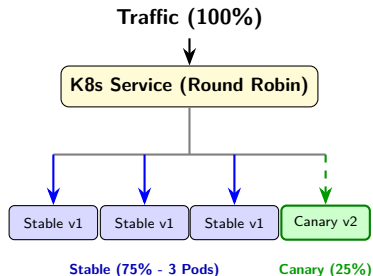
Digest	Tags	Media type	OS/ARCH	Size	Last pushed	Last pulled	Status
sha256:389c4e2c0d1c	v1.1.3-beta1-arm64	Image Index	-	215.9 MB	42 minutes	less than 1 day	Active
sha256:80f0f27497a9	v1.1.3-beta1	Image Index	-	434.6 MB	41 minutes	less than 1 day	Active
sha256:dfe1ce9812ad	v1.1.3-beta1-arm64	Image Index	-	218.7 MB	42 minutes	less than 1 day	Active
sha256:cb59ab04facf	prod v1.1.3	Image Index	-	434.6 MB	42 minutes	less than 1 day	Active
sha256:c6757412db8f	prod-arm64 v1.1.3-arm64	Image Index	-	218.7 MB	44 minutes	less than 1 day	Active
sha256:9e80b5805102	prod-arm64 v1.1.3-arm64	Image Index	-	215.9 MB	44 minutes	less than 1 day	Active
sha256:583cdf902964		Image	-	2.8 kB	42 minutes	less than 1 day	Active
sha256:c3bc8b522680		Image	linux/arm64	215.9 MB	42 minutes	less than 1 day	Active
sha256:8776976d3a33		Image	-	2.5 kB	42 minutes	less than 1 day	Active
sha256:ace0c68feef		Image	linux/arm64	218.7 MB	42 minutes	less than 1 day	Active

[Upgrade plan](#)

11. CI/CD & Deployment — Canary Deployment

Chiến lược Canary

- Khi có tính năng rủi ro cao, ta có thể triển khai **Canary release**.
- Chạy song song 2 bản Deployment (stable và canary) chia sẻ chung cùng 1 Label Selector.
- K8s Service sẽ phân bổ traffic **round-robin** đều cho tất cả các Pod.
- Tỷ lệ chia traffic phụ thuộc vào tỷ lệ số lượng Pod (VD: 3 Stable Pods + 1 Canary Pod $\approx$ 75%/25%).



12. Kết luận — Tổng kết (1/2) - Đã triển khai

Kiến trúc & Backend

- ✓ DB-per-service (6 PostgreSQL + Redis + MinIO)
- ✓ Transactional Outbox (auth, message services)
- ✓ CQRS (message-service InboxProjector)
- ✓ Event-driven (RabbitMQ/Kafka switchable)
- ✓ gRPC sync + Circuit Breaker (Resilience4j)
- ✓ JWT HS256 auth (không cần OAuth2/Keycloak)

Hạ tầng & Vận hành

- ✓ Kong API Gateway (JWT, rate-limit, CORS, Lua)
- ✓ ConfigMap + SOPS-encrypted Secrets
- ✓ Multi-stage Dockerfile, Docker Compose
- ✓ Kubernetes (k3s, Kustomize)
- ✓ CI/CD GitHub Actions → Docker Hub → GKE / k3s
- ✓ Prometheus, Grafana, Loki (Logging)
- ✓ OpenTelemetry + Jaeger (Tracing)
- ✓ Swagger/OpenAPI aggregation

12. Kết luận — Tổng kết (2/2) - Hạn chế & Tương lai

Chưa triển khai / Không phù hợp

- Saga Pattern — không có distributed transaction phức tạp
- OAuth2/Keycloak — không phù hợp: closed system

Hướng phát triển tương lai

- Mở rộng tính năng gọi thoại (Voice Call) / gọi video (Video Call).
- Triển khai Service Mesh (Istio) khi số lượng service tăng lên.

**Cảm ơn thầy và các bạn
đã lắng nghe!**